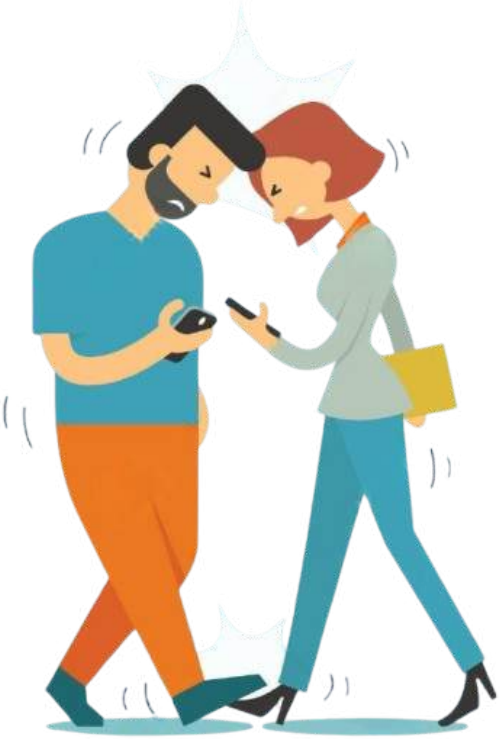
LINK2U



Red social Móvil para conexiones Cercanas mediante bluetooth y ubicación

Marcos Cózar Esteban
Hugo Agustín Zazo

Red social Móvil para conexiones Cercanas mediante bluetooth y ubicación **1**
1. JUSTIFICACIÓN DEL PROYECTO **3**
2. Objetivos **4**
3. Introducción **5**
3.1. Android Studio y Jetpack Compose 5
3.2. Firebase (Realtime Database y Authentication) 5
Firebase ha sido nuestro almacén de datos el cual nos ha permitido tanto volcar cómo recuperar información en todas nuestras operaciones, las cuales nos llevaban a gestionar de manera segura los datos de los usuarios. 6
3.3. Supabase 6
3.4. GitHub 6
3.5. Nearby API 7
3.6. Figma 7
4. Metodología y desarrollo del proyecto **7**
4.1. Enfoque metodológico general 7
4.2. Diseño de la app y estructura inicial 8
4.3. Integración con Firebase y autenticación 9
4.4. Creación y edición de perfiles 13
4.4.1. Modificación de la imagen: 14
4.4.2. Modificación de datos de usuario: Nombre de usuario y redes sociales. 15
4.5. Búsqueda de usuarios cercanos 18
4.5.1. Uso de Nearby API 18
4.5.2. Solicitud de permisos 20
4.5.3. Menú principal y búsqueda de usuarios 22
4.5.4. Usuario encontrado y visión de su perfil 23
4.6. Amigos: Solicitudes, rechazos y lista de amigos 24
4.6.1. Solicitudes de amistad 24
4.6.2. Notificación de la solicitud de amistad 25
4.6.3. Menú de solicitudes: Aceptar o rechazar 25
4.6.4. Lista de amigos 26
4.7. Chats, intercambio de mensajes entre usuarios 29
4.7.1. Chats, envío y recibo de mensajes 29
4.7.2. Notificación de mensajes 31
4.7.3. Borrado de chats 32
5. Resultados y discusión **33**
5.1. Resultado final del desarrollo 33
5.2. Pruebas realizadas 35
5.2.1. Pruebas funcionales: 35
5.2.2. Pruebas de integración: 35
5.2.3. Pruebas de compatibilidad y dispositivos: 35
5.2.4. Pruebas de usabilidad con usuarios reales: 36
5.2.5. Pruebas de errores y casos límite: 36
5.3. Conclusión del apartado 36
6. Conclusiones **37**
6.1. Cumplimiento de los objetivos 37
6.2. Evolución del proyecto y adaptación del anteproyecto 37
6.3. Valor técnico y aprendizaje 38
6.4. Limitaciones y posibles mejoras 38
6.5. Conclusión final 39

1. JUSTIFICACIÓN DEL PROYECTO

Decidimos llevar a cabo este proyecto porque creemos que es una idea interesante, útil y bastante diferente de lo que se ha desarrollado hasta ahora en aplicaciones sociales. Hoy en día, todo el mundo utiliza redes sociales como Instagram, TikTok o WhatsApp, pero todas ellas requieren que hayas conocido previamente a alguien o que exista algún tipo de conexión previa. Eso es justo lo contrario de lo que queremos con esta app: fomentar el encuentro entre personas en la vida real sin que tengan que pasar antes por medios tradicionales ni pedir directamente un número de teléfono. Imagina escenarios como en un festival, en clase, en una biblioteca o en una cafetería, y te encuentras con alguien interesante. Imagina una app que te permita ver que esa persona está allí, ver su perfil y conectar con ella, todo sin invadir su privacidad ni generar una situación incómoda.

También pensamos que es una buena forma de aprovechar tecnologías que muchas veces no se combinan, concretamente BLE (Bluetooth Low Energy) y la localización GPS, dándoles un nuevo propósito. Nos dimos cuenta de que un sistema así encajaría muy bien en eventos como convenciones, universidades o incluso en el transporte público, cuando estás rodeado de posibles personas con quienes conversar pero no siempre sabes cómo iniciar la charla.

Otra razón por la que elegimos este proyecto es porque encaja perfectamente con lo que nos han enseñado en el grado durante los últimos dos años. Queríamos trabajar en algo que nos inspirara y que a la vez sirviera como práctica real. Con esta aplicación podemos usar Firebase para autenticar usuarios y guardar sus datos; Jetpack Compose para crear una interfaz moderna y amigable; Nearby API para detectar usuarios en las cercanías.

Además, nos gusta la idea de implementar las medidas necesarias para proteger la información de los usuarios. Hoy en día, la privacidad es muy importante, y consideramos vital que las personas que usen la app puedan elegir qué compartir y qué no. Por ejemplo, en el perfil podrán poner una foto, una pequeña descripción y enlaces a sus redes sociales si quieren compartirlos, pero siempre con la opción de ocultar lo que no deseen mostrar.

La motivación personal que nos impulsa a realizar este proyecto también tiene mucho peso. Queremos que sea algo nuestro, algo de lo que podamos sentirnos orgullosos. No se trata simplemente de aprobar el módulo de Proyecto, sino de terminar el curso sintiendo que hemos creado algo útil, diferente y bien hecho a nivel profesional. Creemos que este TFG es una gran oportunidad no solo para demostrar lo que hemos aprendido, sino también para seguir aprendiendo, especialmente en temas de optimización, pruebas en dispositivos reales y trabajo en equipo.

En resumen, estamos trabajando en este proyecto porque nos motiva, creemos que puede ser útil para otras personas y nos permite aplicar prácticamente todo lo que hemos estudiado. Además, nos gusta el desafío técnico y creemos que es una gran forma de cerrar nuestra etapa formativa con una implementación real, contemporánea y con verdadero potencial de uso.

2. Objetivos

Nuestro objetivo general con este proyecto es intentar desarrollar una pequeña solución a un pequeño problema con el cual nos encontramos hoy en día en nuestra sociedad. A quien no le ha pasado que se ha sentido atraído por alguien que nos hemos encontrado en la calle de manera espontánea, pero luego no volvemos a saber más sobre esta persona, lo cual a veces puede llegar a ser un poco frustrante. Nosotros venimos a intentar proponer una solución para este problema, como objetivo general pretendemos desarrollar esta aplicación la cual te permite conectar con personas que en otra situación nos sería más complicado tener este tipo de acercamientos.

Dejando a un lado el resultado más general o el objetivo más general que queremos, también tenemos otros objetivos más específicos marcados con el fin de llegar al mismo puerto. Uno de nuestros principales objetivos y uno de los más importantes también es desarrollar una interfaz visual e intuitiva para el usuario, es obvio que una mejor interfaz ayuda a que el usuario se sienta más cómodo a la hora de usar la aplicación, al igual que reduce el porcentaje de problemas o casos dubitativos con las distintas funcionalidades de la misma.

Continuamos con el objetivo de implementar una funcionalidad capaz de detectar a los usuarios cercanos a nosotros, los cuales mediante bluetooth y servicios de ubicación lograremos conseguir esta funcionalidad, con un radio personalizado.

Otros de los objetivos e implementaciones que lograremos o queremos implantar es la creación y personalización de perfiles o de la misma interfaz, en la cual podemos establecer nuestras imágenes para que puedan verlas los demás usuarios de la aplicación. No solo podemos recalcar esta información en nuestro perfil también tendremos la oportunidad de dar a ver nuestras redes sociales el nombre de usuario o la fecha de creación de la cuenta. Estos datos entre otros, los conservaremos en un sistema de guardado en la nube mediante Google Firebase, que veremos más adelante más detalladamente.

Por último, un objetivo crucial a la hora de crear o gestionar una aplicación es el control de usuario y la creación de los mismos. Por todos es bien sabido que la seguridad es una parte crucial que se necesita para conservar la consistencia de la aplicación, para ello usaremos Firebase authentication.

3. Introducción

Para llevar a cabo el desarrollo de esta aplicación tipo red social basada en la detección de usuarios cercanos mediante Bluetooth y localización, se han utilizado distintas tecnologías y herramientas que permiten implementar tanto el frontend como el backend de forma eficiente y moderna. A continuación, se describen las principales tecnologías empleadas y su relación directa con los contenidos vistos en la asignatura de "Acceso a Datos" y, especialmente, "Programación de Dispositivos Móviles".

3.1. Android Studio y Jetpack Compose

Android Studio es el entorno de desarrollo oficial para Android, proporcionado por Google. Permite crear aplicaciones móviles mediante herramientas y simuladores muy útiles para el proceso de diseño, programación y prueba. En este proyecto, se ha utilizado junto con Jetpack Compose, el framework moderno de Google para diseñar interfaces de usuario declarativas. Compose permite definir la interfaz de manera más sencilla y reactiva, adaptándose muy bien al tipo de interacción fluida que requiere una app de tipo "Tinder", donde el diseño y la experiencia del usuario son fundamentales.

Esta tecnología se relaciona directamente con la asignatura de "Programación de Dispositivos Móviles", donde se abordan conceptos como la estructura de una aplicación Android, el ciclo de vida de una actividad, navegación entre pantallas, y diseño responsive.

En nuestro proyecto vemos directamente relacionado y como principal contenedor del mismo a Android Studio, ya que ha sido el entorno de desarrollo el cual nos ha acompañado durante todo el proceso y sobre el cual hemos construido nuestra aplicación.

3.2. Firebase (Realtime Database y Authentication)

Firebase es una plataforma de desarrollo de aplicaciones móviles que ofrece distintos servicios en la nube. En este proyecto se han utilizado principalmente:

Firebase Realtime Database, una base de datos NoSQL que permite almacenar y sincronizar datos en tiempo real entre usuarios conectados. Gracias a esta tecnología, la aplicación puede gestionar los perfiles de usuario y las coincidencias (match) sin necesidad de montar un servidor propio.

Firebase Authentication, que permite implementar de forma sencilla un sistema de registro e inicio de sesión mediante correo electrónico y otros métodos. Es fundamental para gestionar usuarios de forma segura y personalizada.

Ambas herramientas están directamente relacionadas con la asignatura de "Acceso a Datos", ya que se trabaja con almacenamiento de información, estructuras de datos en la nube, y sincronización de datos entre cliente y servidor.

Firebase ha sido nuestro almacén de datos el cual nos ha permitido tanto volcar como recuperar información en todas nuestras operaciones, las cuales nos llevaban a gestionar de manera segura los datos de los usuarios.

3.3. Supabase

Supabase es una alternativa de código abierto a Firebase que proporciona un backend completo con base de datos PostgreSQL, autenticación, almacenamiento y funciones en tiempo real. Supabase es considerado como una opción moderna para gestionar la base de datos del backend, permitiendo almacenar la información de los usuarios, sus perfiles, y posibles relaciones entre ellos.

La base de datos relacional de Supabase permite un mayor control sobre las estructuras de datos, claves foráneas y relaciones entre tablas, lo cual guarda estrecha relación con los contenidos vistos en la asignatura de "Acceso a Datos", especialmente en lo relacionado con modelado de bases de datos, consultas SQL y persistencia de información.

Además, al integrarse fácilmente con APIs RESTful y bibliotecas modernas como Retrofit o Ktor en Android, también se vincula con "Programación de Servicios y Procesos", donde se trabajan conceptos como el acceso remoto a datos, consumo de APIs y la arquitectura cliente-servidor.

El uso de Supabase aporta una alternativa profesional al desarrollo backend sin necesidad de configurar servidores propios, aportando rapidez y escalabilidad en la creación de aplicaciones reales.

En nuestro proyecto Supabase ha supuesto un gran alivio a la hora de almacenar las imágenes o ‘Urls’ de las mismas ya que necesitábamos una ruta pública la cual poder almacenar y ver desde cualquier dispositivo. De esta manera cualquier usuario puede ver en la aplicación las fotos de los demás usuarios. De la otra forma, si las almacenamos en local solo se podrían haber visto en el mismo dispositivo del usuario.

3.4. GitHub

GitHub ha sido la plataforma utilizada para el control de versiones y la colaboración durante todo el proceso de desarrollo del proyecto. Gracias a su integración con Android Studio, ha sido posible mantener un repositorio del proyecto actualizado, registrar cada cambio realizado en el código fuente, y recuperar versiones anteriores en caso de errores o regresiones.

GitHub se relaciona directamente con los conocimientos adquiridos en la asignatura de "Entornos de Desarrollo", donde se trabajan herramientas de control de versiones (como Git), la gestión de ramas, fusión de código y trabajo colaborativo en equipos de desarrollo.

Además, la estructura del repositorio, los **‘commits’** con mensajes significativos y la organización del código han permitido una mejor documentación interna del proyecto, facilitando tanto el trabajo individual como la posibilidad de futuras ampliaciones o mantenimiento por parte de otros desarrolladores.

Durante el desarrollo, GitHub ha sido una herramienta imprescindible no solo para la organización del código, sino también para seguir buenas prácticas de desarrollo profesional, simulando entornos reales de trabajo en empresa

3.5. Nearby API

La Nearby API permite descubrir y comunicarse con dispositivos cercanos usando tecnologías como Bluetooth, Wi-Fi y ultrasonidos. En el contexto del proyecto, se utiliza para detectar usuarios próximos sin necesidad de conexión a internet, lo que permite mostrar perfiles cercanos en tiempo real y sin depender exclusivamente de la geolocalización GPS.

Esta API es especialmente útil en el desarrollo de funcionalidades basadas en proximidad y es un complemento avanzado a lo tratado en clase sobre comunicación entre dispositivos y sensores.

3.6. Figma

Figma es una herramienta de diseño de interfaces que permite crear prototipos visuales y colaborar en tiempo real. En este proyecto, se ha utilizado para planificar y definir el aspecto visual de la aplicación antes de empezar a programarla. Gracias a Figma, ha sido posible diseñar pantallas como la de inicio de sesión, el perfil del usuario y la vista de descubrimiento de otros perfiles, manteniendo una coherencia visual y una buena experiencia de usuario.

Aunque Figma no forma parte directa del desarrollo con código, sí está muy relacionado con la asignatura de "Interfaces", donde se trabaja el diseño visual, la usabilidad y la experiencia del usuario. Utilizar Figma en esta fase ayuda a tener una idea clara de cómo debe comportarse la app, facilitando posteriormente su implementación en Jetpack Compose.

Figma no está tan presente en nuestro proyecto como las demás implementaciones, porque a medida que fuimos avanzando nos dimos cuenta que no podíamos implementar tan fácilmente y con el resultado esperado los avances obtenidos a nuestro proyecto en Android Studio. Aun así si que hay parte del pequeño avance en Figma implementado en Android.

4. Metodología y desarrollo del proyecto

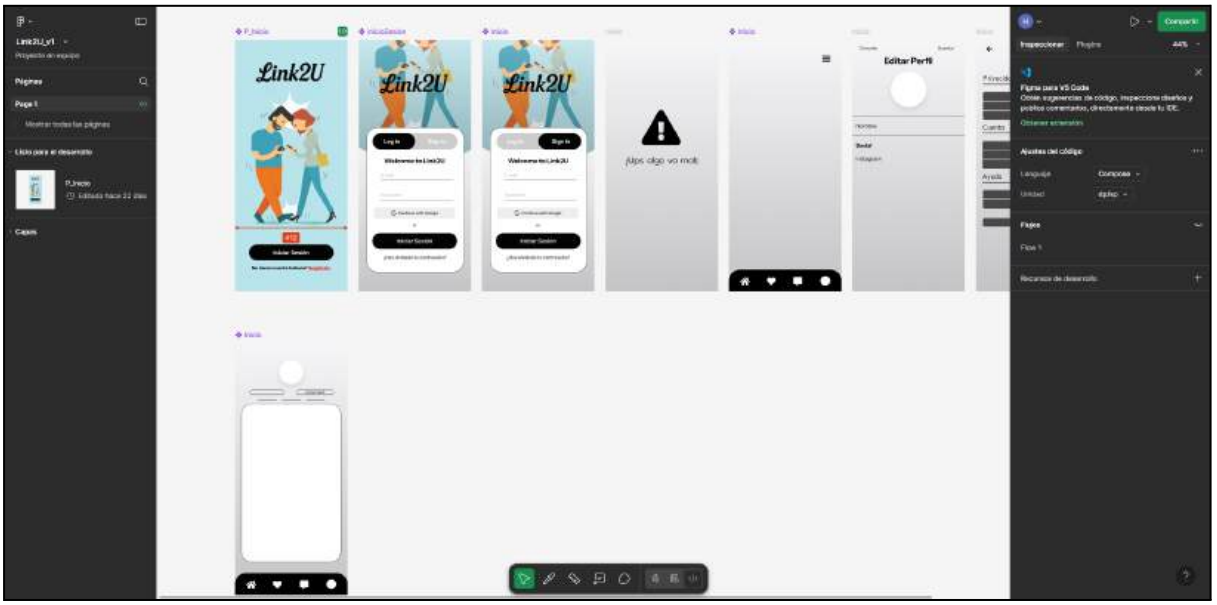
4.1. Enfoque metodológico general

Para llevar a cabo este proyecto seguimos una metodología iterativa, inspirada en enfoques ágiles como Scrum, aunque adaptada a nuestro contexto académico. Dividimos el desarrollo en varias fases semanales, asignando tareas y objetivos claros para cada etapa. Nos apoyamos en herramientas como Trello para la organización del trabajo y GitHub para el control de versiones.

La idea principal fue desarrollar pequeños bloques funcionales e ir construyendo sobre ellos, realizando pruebas constantes y validaciones con usuarios reales (compañeros de clase, amigos y familiares). Esto nos permitió detectar errores rápidamente y adaptar el proyecto a las necesidades reales que iban surgiendo durante el desarrollo.

4.2. Diseño de la app y estructura inicial

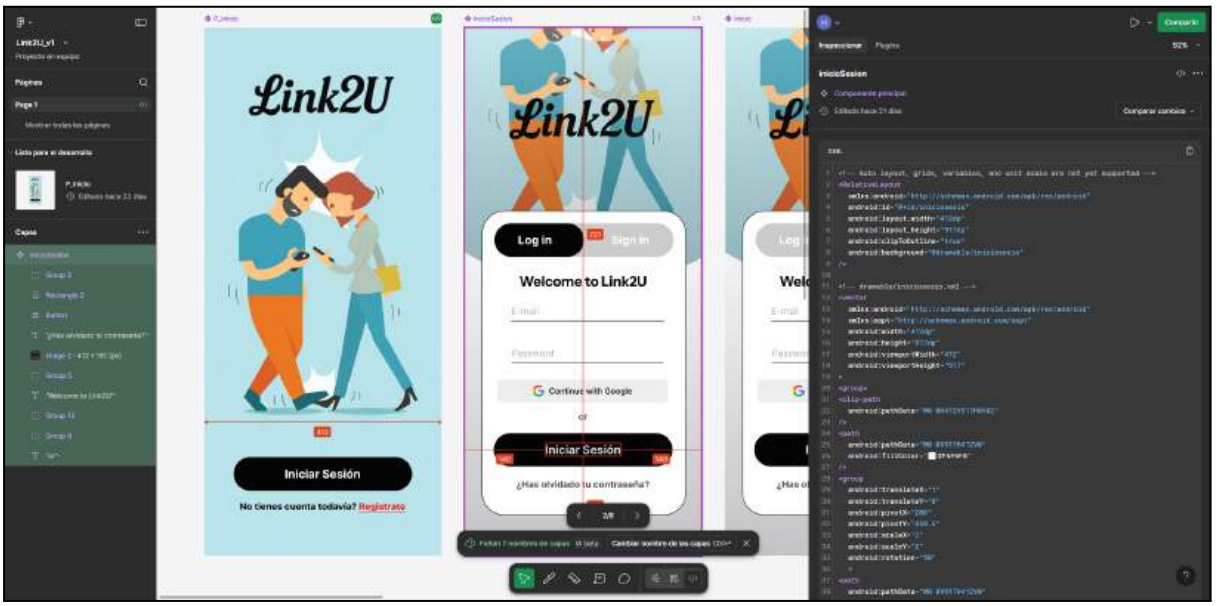
Comenzamos con sesiones de lluvia de ideas para definir qué pantallas tendría la app, cómo se organizaría la navegación, y qué funcionalidades serían imprescindibles desde el principio. Esto se plasmó en bocetos y wireframes usando Figma.



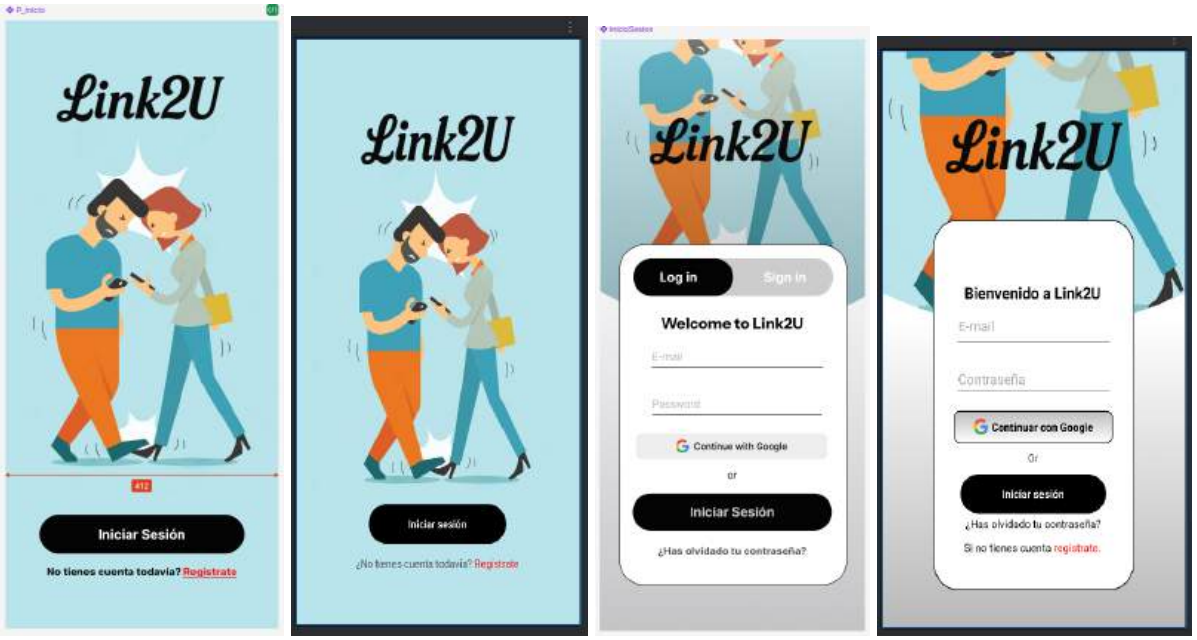
Como se puede observar y como hemos descrito anteriormente, comenzamos este proyecto haciendo estas interfaces en Figma. A la hora de intentar implementarlas estuvimos mucho tiempo intentando importarlas a nuestro proyecto Android, pero al ser un entorno prototipado nos resultó muy difícil ya que no había una compatibilidad directa con Android.

Estuvimos probando con la importación de xml que genera Figma e importarlos posteriormente al proyecto, pero el resultado no era el esperado pues los elementos

de la interfaz estaban descolocados y no seguían el patrón deseado.



Al estrellarnos a la hora de usar estos métodos, seguimos investigando y encontramos que mediante tokens y plugins podíamos hacer una especie de importado y exportado de las interfaces, lo cual probamos y fue la solución que más se acercó al resultado que esperábamos por lo que intentamos adaptar a nuestra manera y acercarlo lo máximo posible a lo que queríamos.



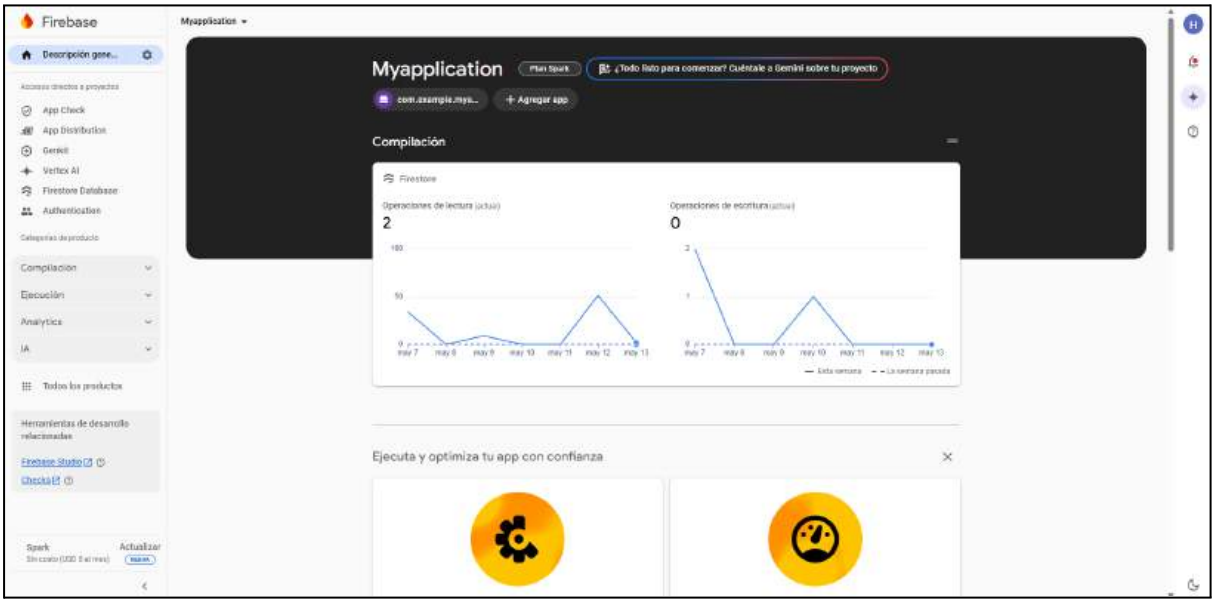
Figma **Android** **Figma** **Android**

Como se puede observar la gran mayoría de interfaces conseguimos replicarlas casi perfectas.

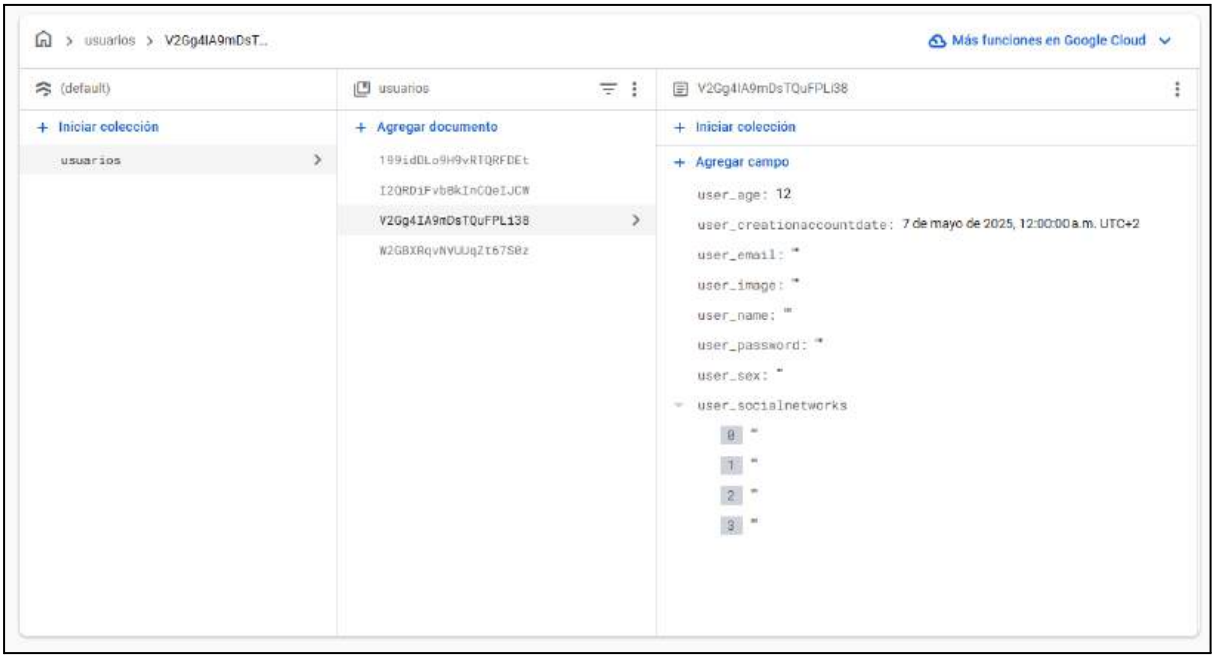
4.3. Integración con Firebase y autenticación

Una vez integradas gran parte de las interfaces nos pusimos manos a la obra con el tema de usuarios y dar de alta a los mismos para ello Firebase fue la base para toda la lógica backend de la aplicación. Primero configuramos Firebase Authentication para permitir el login mediante Google y correo/contraseña.

Una vez autenticado el usuario, se crea un documento asociado en Firestore con su UID, donde se almacenan datos como nombre, descripción, redes sociales, coordenadas de ubicación y enlaces a imágenes.



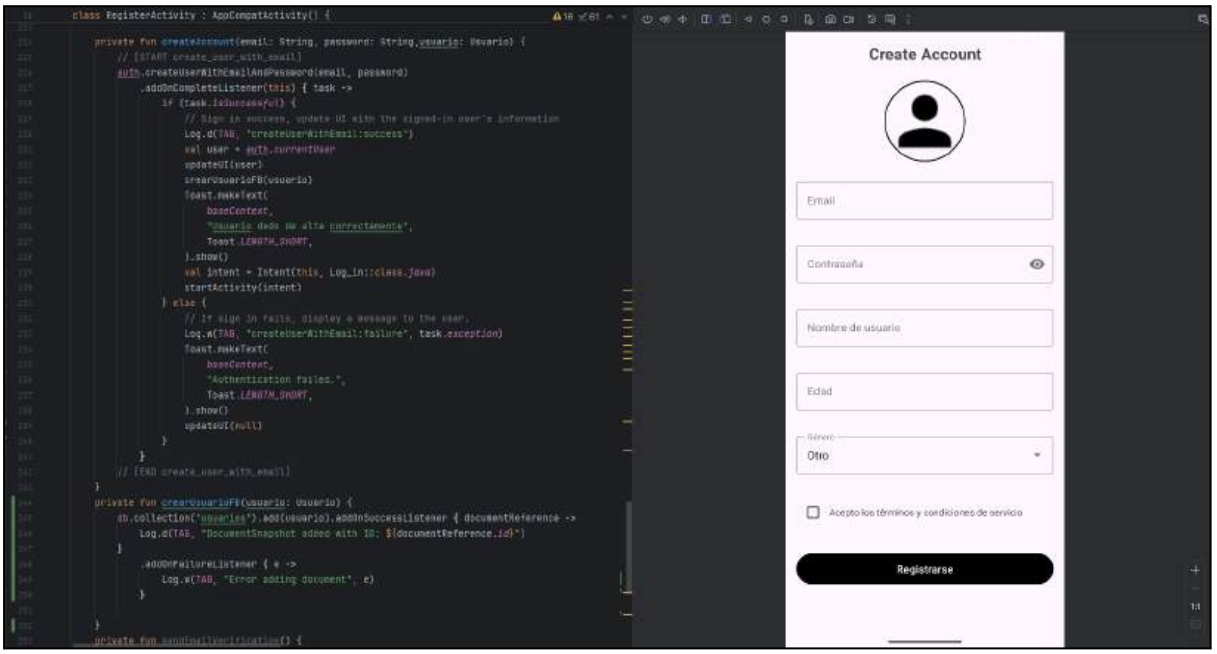
También podemos observar la base de datos de Firestore en la que trabajamos en una base de datos llamada usuarios:



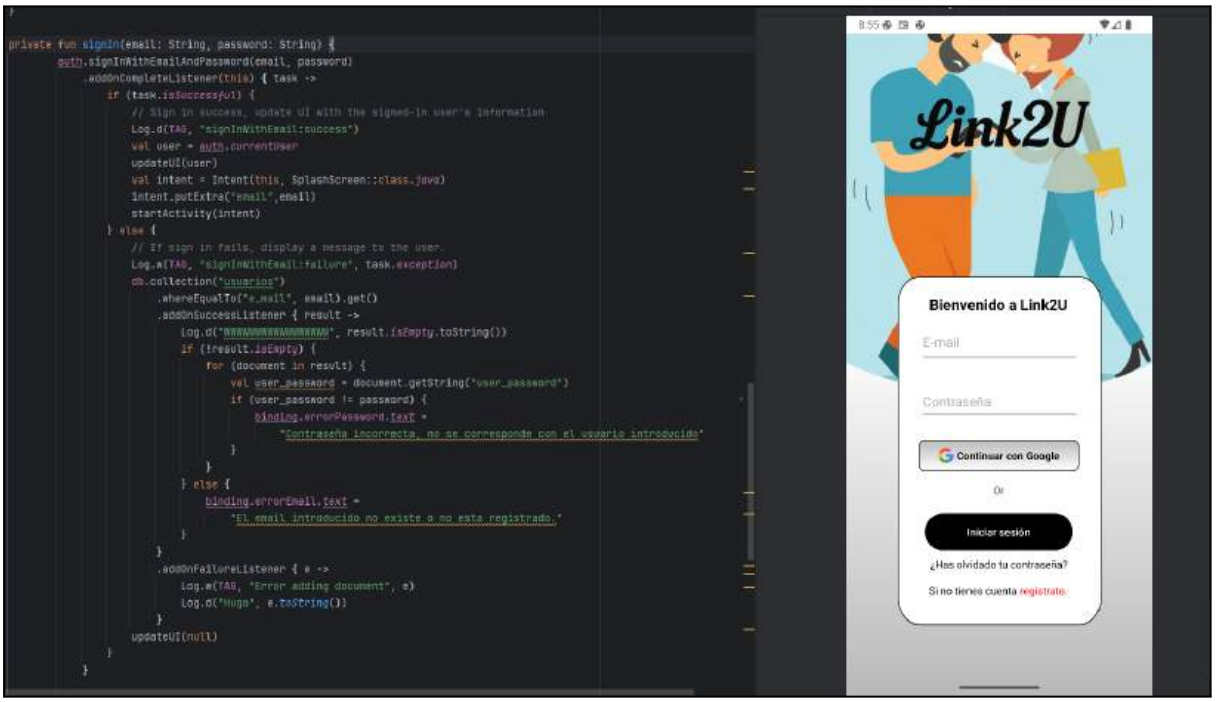
Por último, para obtener una mayor seguridad a la hora de acceder a la aplicación decidimos usar la herramienta de autenticación la cual nos permite elegir qué métodos de autenticación queremos para nuestro proyecto. En nuestro caso elegimos mediante ‘Google’ y mediante ‘Correo y contraseña’:

También se añadieron reglas de seguridad que impiden que un usuario edite el documento de otro, y que solo pueda acceder a sus propias imágenes almacenadas en Firebase Storage.

Las interfaces y código que permiten lo que acabamos de ver son las siguientes:



Como podemos observar esta es la pantalla de registro, la cual se encarga como su propio nombre indica de registrar al usuario tanto a nivel de la base de datos, como a nivel de autenticación. En esta interfaz tenemos unos recuadros los cuales se encargaran de recoger toda la información sobre el usuario, que posteriormente será subida a la nube para dejarla registrada. También hacemos un control de errores como por ejemplo evitar que quede algún campo vacío, que la edad sea menor de 18 años o que la contraseña tenga menos de 6 caracteres. Por último cabe destacar que controlamos que ningún varios usuario tengan el mismo nombre o correo en la cuenta, haciéndolos así campos únicos para evitar tener 100 usuarios con los mismos datos.



Esta interfaz se encarga de controlar el inicio de sesión, en el cual comprobamos que exista el usuario y que coincidan correo y contraseña con lo establecido, si es así pasaremos a nuestra siguiente pantalla.

Otro recurso que hemos implementado y tiene que ver de manera indirecta con el inicio de sesión es la capacidad de guardar el estado de la aplicación. Por ejemplo una vez que ya hemos iniciado bien sesión y ya estamos dentro de nuestra aplicación no es necesario que estemos una y otra vez introduciendo nuestro correo y contraseña para usar la aplicación sería muy poco práctico. Esta herramienta nos permite guardar los datos ya introducidos y comprobar si hay un email ya guardado entonces no es necesario introducir de nuevo las credenciales.

Al igual que podemos guardar las credenciales, también podemos hacer un barrido para que cuando se cierre sesión dar la oportunidad de volver a iniciar de nuevo.Todo esto lo controlamos en distintas estructuras pero las fundamentales son estas dos:

```
private fun session(){
    val prefs = getSharedPreferences("com.example.myapplication.PREFERENCE_FILE_KEY", Context.MODE_PRIVATE)
    val email_recuperado = prefs.getString("email",null)
    if(email_recuperado!=null){
        val intent = Intent(this, SplashScreen::class.java)
        intent.putExtra("email",email_recuperado)
        startActivity(intent)
    }

}
```

```
binding.logoutButton.setOnClickListener {
    val prefs = getSharedPreferences("com.example.myapplication.PREFERENCE_FILE_KEY", Context.MODE_PRIVATE).edit()
    prefs.clear()
    prefs.apply()
    val intent = Intent(this, Log_in::class.java)
    startActivity(intent)
}
```

En esta última limpiamos las credenciales para que la próxima vez que se inicie sesión se guarden las siguientes.

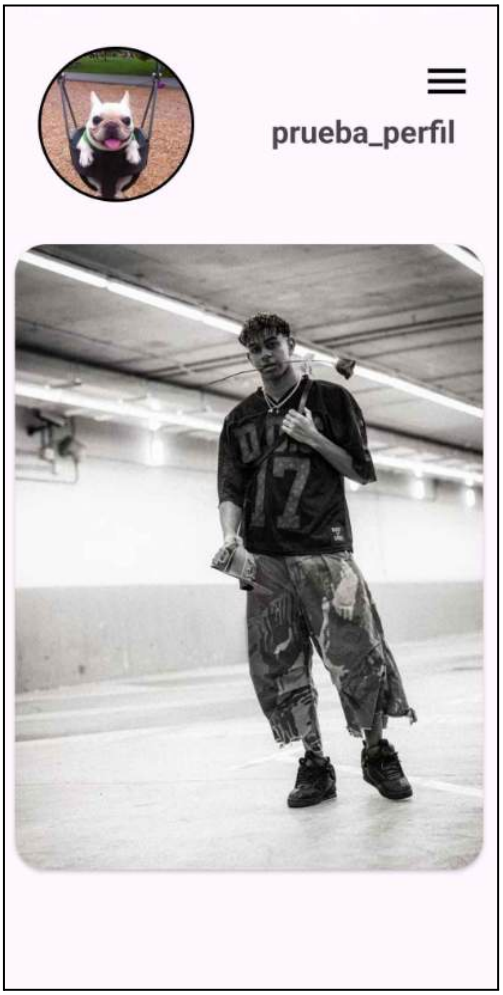
4.4. Creación y edición de perfiles

Cada usuario puede editar su perfil desde una sección específica. Esta información se sincroniza en tiempo real con Firebase Firestore.

Para la galería de imágenes usamos una base de datos llamada Supabase. Añadimos una función de compresión de imágenes antes de la subida para reducir el peso y mejorar el rendimiento de carga. Cada usuario al registrarse elige una foto de perfil, que posteriormente podrá modificar.

Para acudir al perfil y así a su configuración y subida de imágenes se deberá pulsar el icono de usuario, ubicado en la parte inferior de la barra de acción del menú principal:

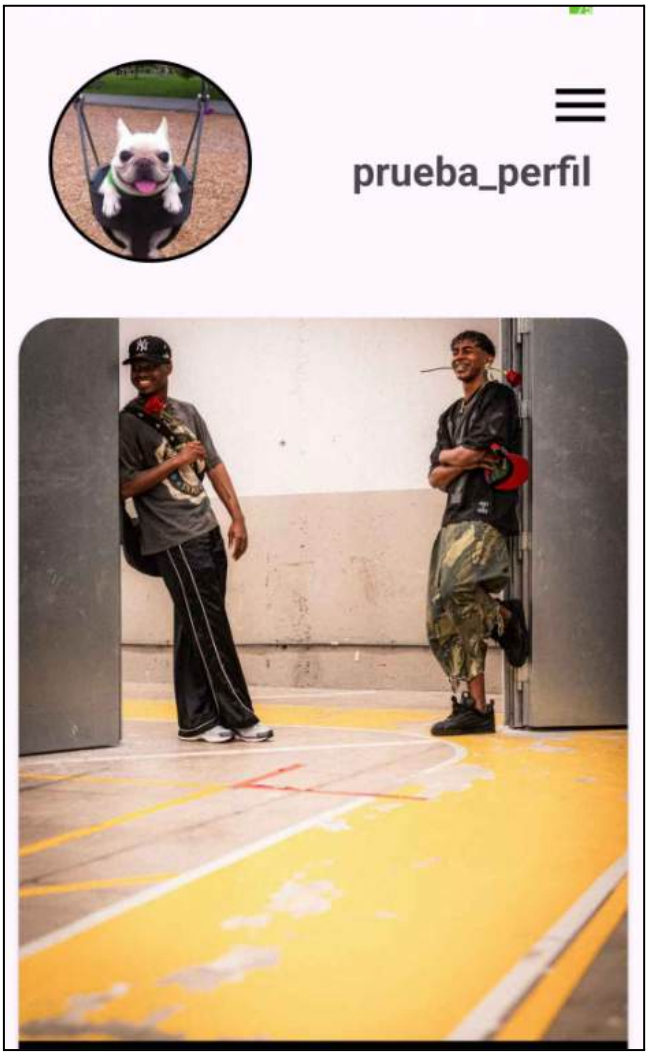
Una vez en este fragmento, tendremos a nuestra disposición cuatro elementos: Foto de perfil, las tres imágenes editables, nuestro nombre de usuario y un menú hamburguesa en la esquina superior derecha.



4.4.1. Modificación de la imagen:

Para las modificaciones de las imágenes de presentaciones, bastará con dar un toque en cada una de las imágenes, lo que abrirá una pequeña ventana emergente que solicitará la elección de uno de los métodos para subir fotos.

Al abrir la galería y elegir una imagen, ésta sustituirá a la que está puesta o añadirá a los huecos que hay en blanco. Aquí un ejemplo de sustitución de imágenes:



Se podrá hacer esto hasta con las tres imágenes deslizables que existen en el perfil.

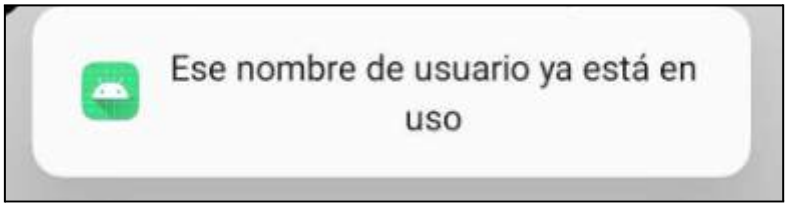
4.4.2. Modificación de datos de usuario: Nombre de usuario y redes sociales.

Al pulsar el botón hamburguesa ubicado en la esquina superior derecha, se nos ubicará en una pantalla donde tendremos acceso a los datos de nuestra cuenta con varios datos:

Aquí tendremos todos los datos importantes de nuestra cuenta, los cuales podemos modificar para su posterior uso tocando en el texto “Editar” de ambas secciones (Cuenta y Redes Sociales):

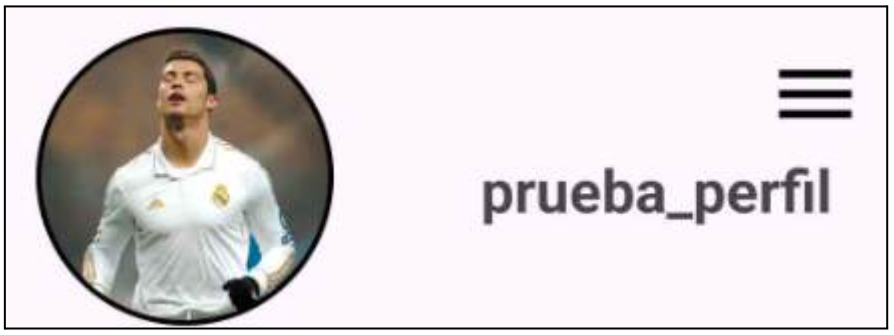
Los campos a los que da acceso son los editables. Nombre de usuario y redes sociales. Si se desean guardar los datos, basta con pulsar el botón “Guardar” para almacenarlos en la base de datos. Si se da a “Cancelar”, estos serán descartados y por ende, no almacenados.

El nombre de usuario, al poder estar repetido, tiene un control, que no te permite cambiar a un nombre ya existente en la base de datos para evitar conflictos.



El botón “Cerrar sesión”, como su nombre indica, cerrará la sesión del usuario activo y nos devolverá al menú de inicio de sesión para volver a iniciar sesión con el usuario que se deseé.

Arriba del perfil, se dispondrá de la imagen de usuario, la cual es modificable, basta con tocar la imágen para poder modificarla. Se realiza exactamente igual que las de perfil y se mostrará en el perfil



4.5. Búsqueda de usuarios cercanos

Esta fue una de las partes más complejas. Con el uso de Nearby API de Google, conseguimos realizar conexiones entre dos dispositivos que estén cercanos entre 10 y 100 metros dependiendo del entorno, interferencias, y si se están utilizando tecnologías como Bluetooth 5.0 o WiFi.

4.5.1. Uso de Nearby API

Para la conexión de dos dispositivos con Nearby API, es necesario que estos dos dispositivos estén ambos en los estados “Discovery” (descubriendo) y “Advertising” (Anunciándose). Estos son los dos estados que maneja Nearby API para la conexión de los dispositivos.

Lo que hemos realizado en código ha sido poner el dispositivo tanto en Discovery como en Advertising. Así, siempre garantizamos una conexión al no tener únicamente un modo disponible y esperar a que otro usuario tuviera el modo contrario para realizar la conexión.

En la siguiente imagen se muestra la parte de codigo donde se inicia el modo “Advertising”, que será el responsable de mostrar nuestro dispositivo a otros.

```
fun startAdvertising(context: Context) {
    currentContext = context
    if (isAdvertising) return
    isAdvertising = true
    _connectionState.value = ConnectionState.ADVERTISING

    val advertisingOptions = AdvertisingOptions.Builder()
        .setStrategy(Strategy.P2P_CLUSTER)
        .build()

    Nearby.getConnectionsClient(context).startAdvertising(
        currentUsername ?: "Unknown User",
        SERVICE_ID,
        connectionLifecycleCallback,
        advertisingOptions
    ).addOnSuccessListener {
        Log.d(TAG, msg: "Advertising started successfully")
        startReconnectLoop(context)
    }.addOnFailureListener {
        Log.e(TAG, msg: "Failed to start advertising", it)
        _connectionState.value = ConnectionState.ERROR
        isAdvertising = false
    }
}
```

En esta imagen se muestra cómo empieza el otro estado, el “Discovery”. Este será el responsable de encontrar otros dispositivos y realizar la conexión.

```
fun startDiscovery(context: Context) {
    currentContext = context
    if (isDiscovering) return
    isDiscovering = true
    _connectionState.value = ConnectionState.DISCOVERING

    val discoveryOptions = DiscoveryOptions.Builder()
        .setStrategy(Strategy.P2P_CLUSTER)
        .build()

    Nearby.getConnectionsClient(context).startDiscovery(
        SERVICE_ID,
        endpointDiscoveryCallback,
        discoveryOptions
    ).addOnSuccessListener {
        Log.d(TAG, msg: "Discovery started successfully")
    }.addOnFailureListener {
        Log.e(TAG, msg: "Failed to start discovery", it)
        _connectionState.value = ConnectionState.ERROR
        isDiscovering = false
    }
}
```

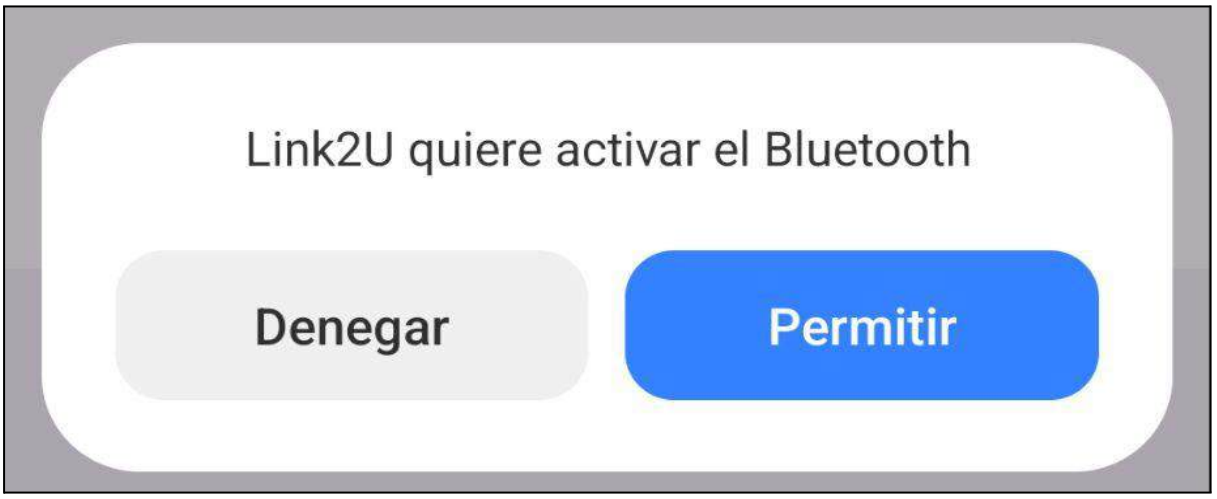
Aquí tenemos otra parte del código que será el responsable de que estos dos estados, mientras la aplicación está activa, esten constantemente refrescandose, para realizar conexiones de manera continua y no perder conectividad con los usuarios que ya se han encontrado.

```
private fun startReconnectLoop(context: Context) {
    reconnectJob?.cancel()
    reconnectJob = viewModelScope.launch {
        while (true) {
            delay( timeMillis: 30000) // Reintentar cada 30 segundos
            if (!isAdvertising) {
                startAdvertising(context)
            }
            if (!isDiscovering) {
                startDiscovery(context)
            }
        }
    }
}
```

4.5.2. Solicitud de permisos

Antes de iniciar el menú donde se realizará la búsqueda de usuarios, se obliga al usuario a encender las tecnologías necesarias para el correcto funcionamiento: Bluetooth y ubicación. Se mostrará una pequeña ventana que preguntará al usuario si quiere activar estos permisos. Si son declinados, saltará una ventana de error que recomienda al usuario reiniciar la aplicación y volver a iniciar sesión para, esta vez sí, conceda los permisos correspondientes y active las tecnologías a usar.

Aquí se muestra el permiso de solicitud de Bluetooth:



Una vez aceptado el permiso de Bluetooth, se procede a solicitar el permiso de ubicación. Te llevará directo a los permisos de la aplicación donde se tendrá que activar la opción “Usar ubicación”.

Si no se acepta alguno de los permisos anteriores, saltará esta pantalla de error solicitando el reinicio de la aplicación para que se vuelvan a conceder los permisos necesarios para el correcto funcionamiento.

4.5.3. Menú principal y búsqueda de usuarios

Una vez dados los permisos, se empezará con la búsqueda de usuarios. Un pequeño texto arriba de la pantalla avisará de en qué estado está, ya que puede “Buscar usuarios”, estar “Conectado” o “Desconectado” o tener un “Error de conexión”.

Aquí una imagen de como es el menú principal:

Estando en la pantalla inicial, comenzará la búsqueda de usuarios. Una vez conectado al otro usuario, su perfil aparecerá por pantalla (una lista RecyclerView) donde se mostrará tanto su foto de perfil como su nombre de usuario.

Aquí se muestra una imagen cuando se realiza la conexión con el usuario:

4.5.4. Usuario encontrado y visión de su perfil

Una vez pulsemos a dicha persona nos saltará un activity donde podremos ver su perfil y tener acceso a datos como su edad y su sexo, aparte de las 3 imágenes que puede configurar para su perfil. En el medio se dispondrá de un botón “Añadir amigo”, el cual le mandará una solicitud al usuario contrario sobre qué se quiere entablar una amistad con él/ella.

4.6. Amigos: Solicitudes, rechazos y lista de amigos

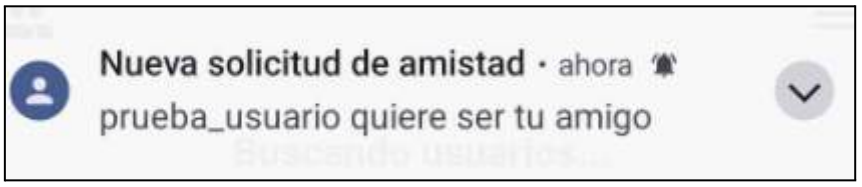
4.6.1. Solicitudes de amistad

Una vez enviada la solicitud de amistad, ésta quedará almacenada en la base de datos, vinculada al perfil del usuario que la ha recibido. El destinatario podrá ver esta solicitud dentro de una sección de notificaciones o solicitudes pendientes. Desde ahí, podrá elegir entre aceptar o ignorar la petición.

Aquí una imagen de lo que sucede cuando se pulsa el botón de solicitud de amistad:

4.6.2. Notificación de la solicitud de amistad

Cuando un usuario te envia una solicitud, recibirás la siguiente notificación, avisando de que tienes una solicitud de amistad:



4.6.3. Menú de solicitudes: Aceptar o rechazar

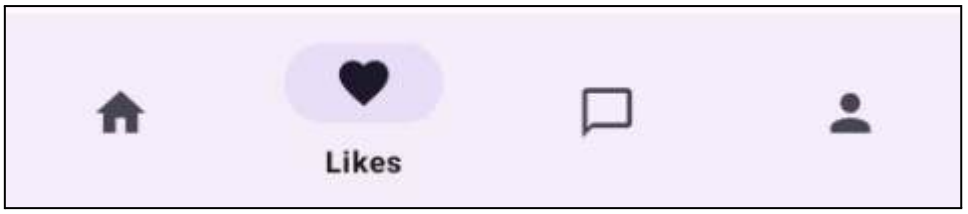
Cuando recibas la notificación, deberás acudir al siguiente icono (ubicado en el menú principal)

Una vez pulsado el icono, entrarás a un activity donde podrás aceptar o rechazar las solicitudes de amistad. Si se rechazan, se podrá volver a enviar solicitud a dicho usuario.

4.6.4. Lista de amigos

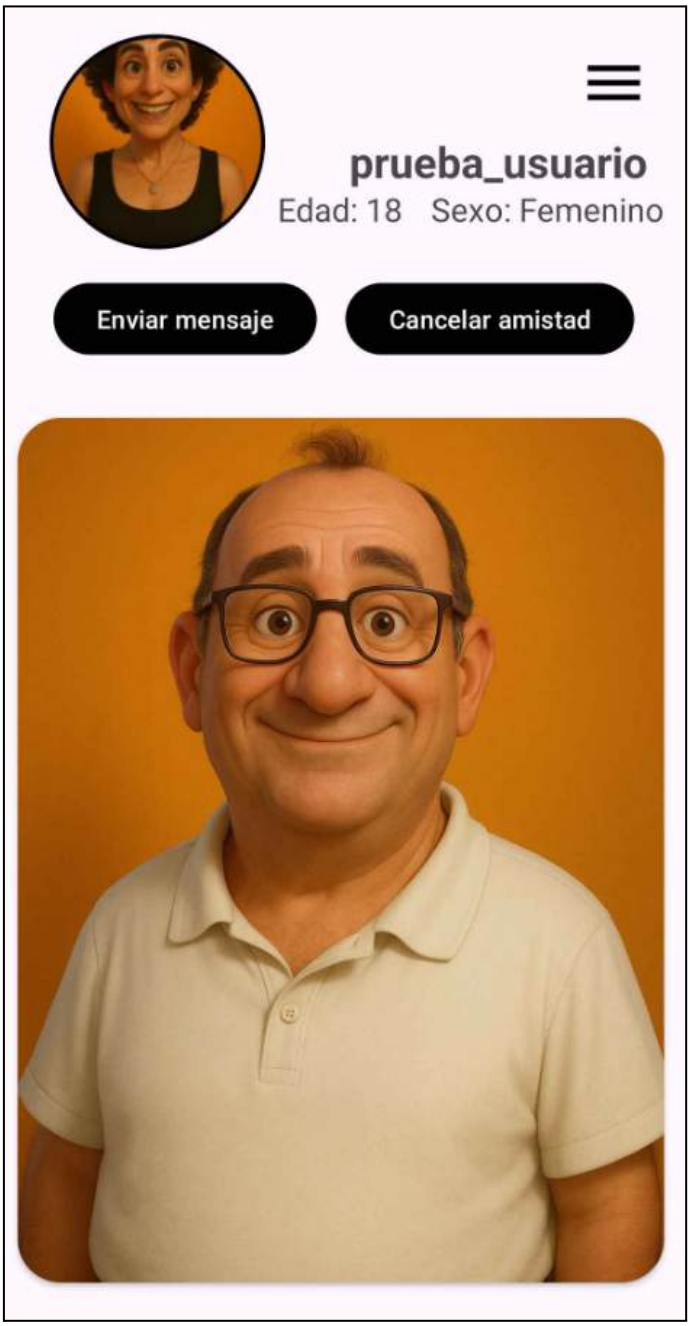
Si decide aceptarla, ambos usuarios pasarán a formar parte de la lista de amigos del otro. Esta lista se cargará automáticamente en el fragmento llamado “Likes”, donde aparecerán de forma ordenada todos los contactos con los que se ha establecido una relación de amistad.

Fragmento donde encontrar amistades:

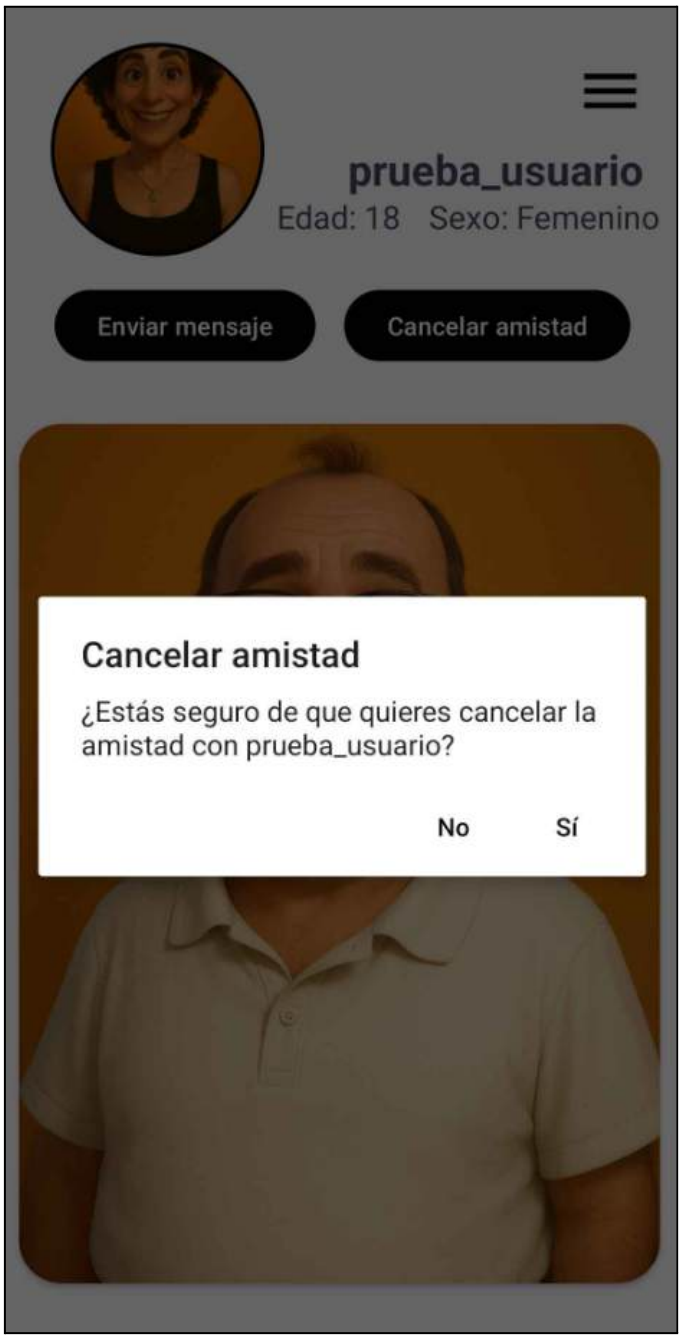


Cada amigo dentro de esta sección contará también con una pequeña vista previa, mostrando nuevamente su imagen de perfil y su nombre de usuario.

Al pulsar el nombre del usuario, se entrará a una actividad donde se muestra el perfil de tu amigo con dos opciones. Enviar mensaje o Cancelar amistad. Aparte de las tres rayas en la parte superior derecha.



Si se pulsa el botón de cancelar amistad, se muestra una ventana emergente preguntando si estamos seguros de cancelar la amistad. Si le decimos que no, el usuario se mantendrá en nuestra lista de amigos y en su fragmento correspondiente. Si se pulsa sí, ese usuario se eliminará de nuestros amigos y no aparecerá más:



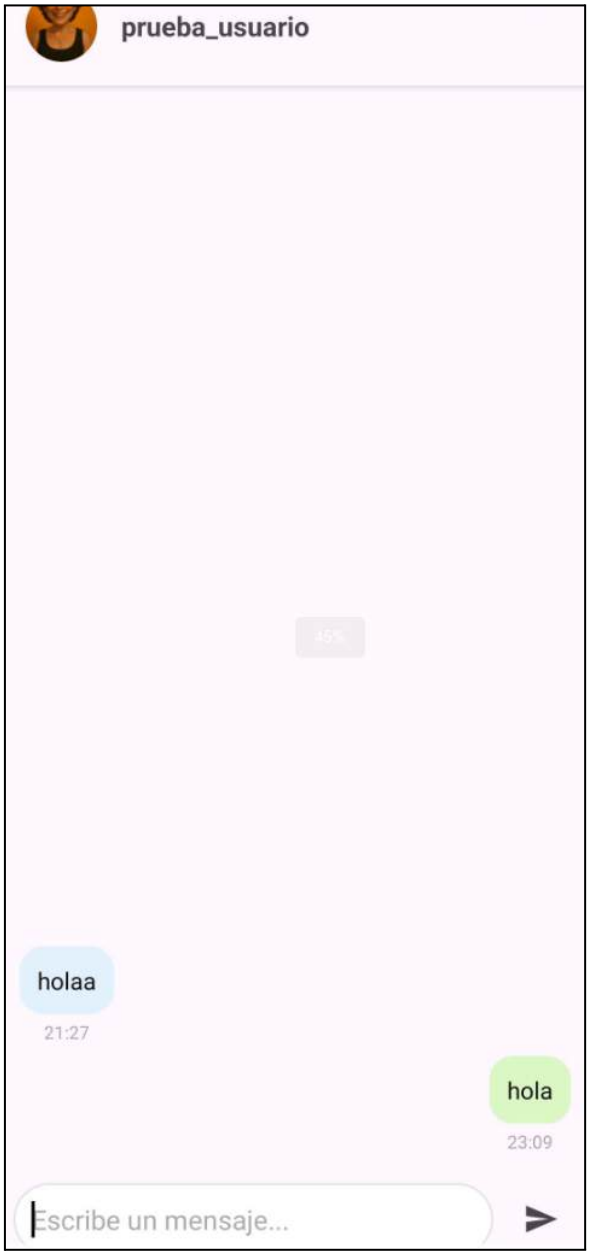
Si pulsamos el botón superior derecha, iremos a una actividad donde podremos ver los datos de nuestro amigo, como su correo electrónico, la fecha de creación de la cuenta, su edad y sus redes sociales.

Con este sistema hemos conseguido que la conexión entre usuarios sea sencilla, visual y accesible, evitando complicaciones y haciendo uso eficiente de las herramientas que ofrece Nearby API junto con Firebase.

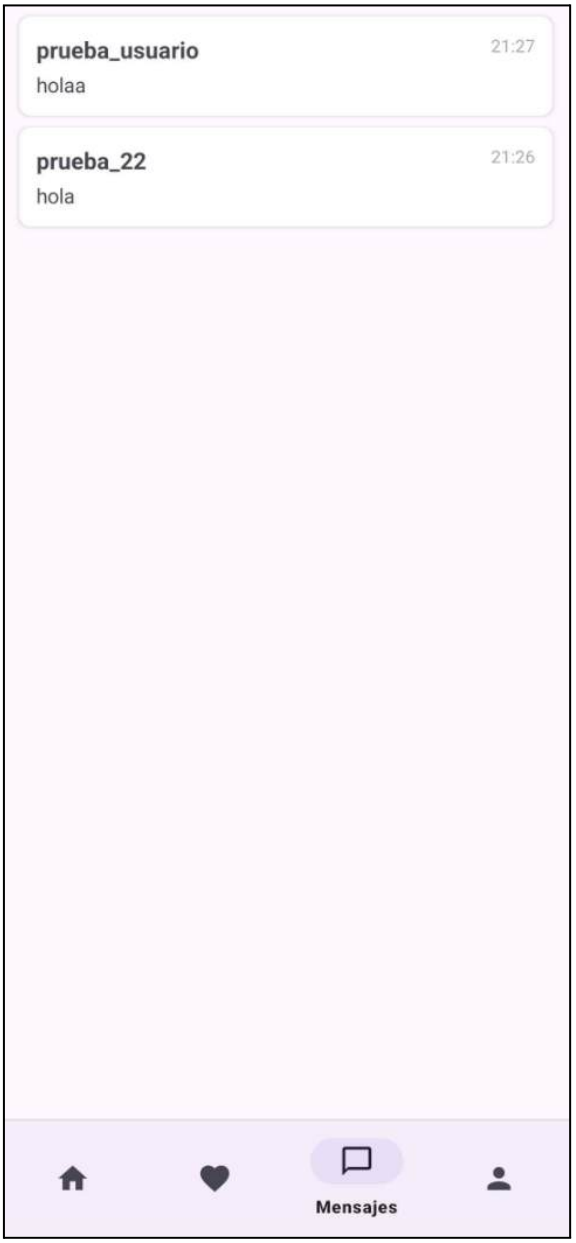
4.7. Chats, intercambio de mensajes entre usuarios

4.7.1. Chats, envío y recibo de mensajes

Al aceptar a una persona, en su perfil, en el botón de “Enviar mensaje”, se podrá acceder al chat con la persona seleccionada. Se trata de una actividad donde se pueden compartir mensajes en tiempo real con el usuario añadido como amigo.



Otra forma de acceder a los chats es mediante el fragmento “Mensajes”, ubicado en la parte inferior de la pantalla principal donde residirá una lista de los chats que tiene cada usuario con sus amigos. Al pulsar en cada uno de ellos se entrará al chat con el respectivo usuario amigo.



4.7.2. Notificación de mensajes

También existen las notificaciones de mensajes: Cada vez que recibes un mensaje nuevo, en la parte superior de la pantalla aparecerá el mensaje acompañado con el nombre del usuario que ha enviado dicho mensaje:

4.7.3. Borrado de chats

Los chats se pueden mantener únicamente con amigos, por ello, al cancelar la amistad, se borrarán automáticamente también los chats que mantienes con ellos:

```
private fun cancelFriendship() {
        .addOnSuccessListener { result2 ->
                .addOnSuccessListener { documents ->
                    // Eliminar el primer chat y sus mensajes
                    db.collection( collectionPath: "chats").document(chatId1)
                        .collection( collectionPath: "messages")
                        .get()
                        .addOnSuccessListener { messages ->
                            messages.forEach { message ->
                                message.reference.delete()
                            }
                            db.collection( collectionPath: "chats").document(chatId1).delete()
                        }

                    // Eliminar el segundo chat y sus mensajes
                    db.collection( collectionPath: "chats").document(chatId2)
                        .collection( collectionPath: "messages")
                        .get()
                        .addOnSuccessListener { messages ->
                            messages.forEach { message ->
                                message.reference.delete()
                            }
                            db.collection( collectionPath: "chats").document(chatId2).delete()
                            .addOnSuccessListener {
                                    Toast.makeText( context: this,  text: "Amistad cancelada", Toast.LENGTH_SHORT).show()
                                    hideFriendButtons()
                                    finish()
                                }
                        }
                }
        }
```

5. Resultados y discusión

5.1. Resultado final del desarrollo

El desarrollo del proyecto ha culminado en una aplicación móvil completamente funcional, cuyo objetivo principal, facilitar la conexión entre personas cercanas mediante tecnologías como Bluetooth y localización, se ha logrado con éxito. A lo largo del proceso se han desarrollado e integrado una serie de funcionalidades clave que responden a los objetivos generales y específicos establecidos al inicio de la memoria.

A continuación, se describen y analizan los principales resultados técnicos obtenidos:

- **Sistema de autenticación de usuarios:** Se implementó utilizando Firebase Authentication, permitiendo a los usuarios registrarse y acceder mediante correo electrónico o cuenta de Google. Se aplicaron validaciones estrictas para garantizar la integridad de los datos (edad mínima de 18 años, contraseña mínima de 6 caracteres, campos obligatorios, verificación de duplicados, etc.). Se logró además mantener la sesión activa sin necesidad de repetir credenciales en cada inicio de aplicación, lo que mejora notablemente la experiencia de usuario.

- **Creación y edición de perfiles personalizados:** Los usuarios pueden configurar su perfil con nombre, edad, redes sociales, sexo y hasta tres imágenes personales. Se integró una lógica para impedir nombres de usuario

repetidos y se estableció un sistema eficiente de almacenamiento de imágenes en Supabase, lo que permite que estas estén disponibles públicamente desde cualquier dispositivo, mejorando el acceso y reduciendo el peso mediante compresión previa.

- **Detección de usuarios cercanos:** Una de las funcionalidades más importantes y complejas del proyecto ha sido la implementación de la Nearby API de Google. Esta API permite detectar usuarios dentro de un rango físico real (aproximadamente entre 10 y 100 metros, dependiendo del entorno). Se configuró la aplicación para que los dispositivos estén continuamente en modo *Advertising* y *Discovery*, permitiendo conexiones bidireccionales. Esta lógica asegura que cualquier usuario que entre en el rango sea detectado y añadido a la lista visual de descubrimiento. Esta funcionalidad fue ampliamente testeada y ha demostrado gran fiabilidad, especialmente en entornos controlados como aulas o salas cerradas.

- **Sistema de amistad y notificaciones:** Se ha implementado un sistema completo de solicitud y gestión de amistades. Cuando un usuario desea conectar con otro, puede enviarle una solicitud. Esta se registra en la base de datos y se notifica al destinatario. Desde una sección específica, este puede aceptar o rechazar la solicitud. Si la acepta, ambos pasan a ser amigos y su relación queda reflejada en el fragmento "Likes", donde se listan todos los contactos aceptados. Además, se agregó una función para eliminar amigos, la cual también borra automáticamente los chats compartidos, manteniendo la consistencia de los datos.

- **Sistema de mensajería instantánea:** Los usuarios que hayan establecido amistad pueden intercambiar mensajes en tiempo real mediante un chat privado. Esta función fue implementada con Firebase Realtime Database, lo que permite sincronización inmediata. También se incluyó un sistema de notificaciones emergentes para avisar de nuevos mensajes entrantes. Esta función ha demostrado gran utilidad y solidez durante las pruebas.

- **Diseño visual y experiencia de usuario:** Aunque inicialmente se diseñaron prototipos en Figma, la dificultad de exportarlos a Android Studio obligó a rediseñar los elementos manualmente utilizando Jetpack Compose. A pesar de ello, se logró una interfaz fluida, limpia, moderna y coherente, que guía al usuario de manera intuitiva por las distintas secciones de la app. La navegación, los botones, los formularios y los listados están todos diseñados para ser comprensibles incluso sin instrucciones externas.

Funcionalidad	Estado	Pruebas realizadas	Observaciones
Autenticación	Completada	Sí	Registro seguro y validación de campos
Edición de perfiles	Completada	Sí	Imágenes e

			información editable en tiempo real
Detección Nearby	Completada	Sí	Alta precisión en interiores, con conectividad continua
Solicitudes de amistad	Completada	Sí	Flujo de solicitud, notificación y aceptación funcional
Chat entre amigos	Completado	Sí	Interfaz clara, respuesta en tiempo real, eliminación automática
Diseño visual	Completado	Sí	UI adaptada y funcional con Jetpack Compose

5.2. Pruebas realizadas

Con el objetivo de validar la fiabilidad, seguridad y usabilidad de la aplicación, se llevaron a cabo diferentes tipos de pruebas. Estas pruebas permitieron detectar errores, confirmar la robustez de las funcionalidades y ajustar pequeños detalles para mejorar la experiencia final del usuario.

5.2.1. Pruebas funcionales:

Se probaron todos los flujos básicos del usuario: creación de cuenta, login, modificación de perfil, detección de otros usuarios, solicitud de amistad, aceptación/rechazo de solicitudes, inicio de chats, cierre de sesión, etc. Cada prueba fue ejecutada repetidas veces en distintos dispositivos Android. Se confirmó que todas las funciones se comportan de forma consistente y estable.

5.2.2. Pruebas de integración:

Se verificó que los distintos módulos de la aplicación trabajen de manera conjunta sin inconsistencias. Por ejemplo, que al aceptar una amistad, se actualice tanto la lista de amigos como la disponibilidad del chat; o que al cambiar una imagen de perfil, se vea reflejado automáticamente en el RecyclerView del descubrimiento y en el perfil mostrado a otros.

5.2.3. Pruebas de compatibilidad y dispositivos:

La aplicación se instaló y ejecutó en varios terminales Android, con versiones del sistema operativo desde Android 8.1 hasta Android 13. En todos los dispositivos se confirmó el correcto funcionamiento de la Nearby API, la gestión de permisos de ubicación/Bluetooth, y el rendimiento general. Se comprobó también que los

permisos solicitados son claros y aparecen en el momento justo, sin interrumpir el flujo natural de uso.

5.2.4. Pruebas de usabilidad con usuarios reales:

Se invitó a varios usuarios (alumnos, amigos y familiares) a utilizar la aplicación sin ayuda. Se observaron sus comportamientos e impresiones. En general, los usuarios encontraron la app intuitiva y atractiva. Algunos comentaron que la opción de guardar sesión mejora mucho la experiencia. También se detectaron sugerencias visuales menores, como reorganizar la posición de ciertos botones, las cuales fueron aplicadas en fases posteriores.

5.2.5. Pruebas de errores y casos límite:

Se realizaron pruebas controladas para comprobar el comportamiento de la app ante situaciones adversas, como:

- Intentar registrar una cuenta con datos inválidos.
- Rechazar los permisos requeridos por Nearby.
- Apagar el Bluetooth o la ubicación durante una búsqueda.
- Cerrar sesión abruptamente o perder conexión a internet.

En todos los casos, la aplicación mostró mensajes de error adecuados, y logró recuperarse sin bloqueos ni pérdida de datos, lo que demuestra una buena gestión de excepciones.

5.3. Conclusión del apartado

Los resultados obtenidos reflejan un trabajo sólido, funcional y bien estructurado. Se ha conseguido desarrollar una aplicación social original, que responde a una necesidad real de conexión en el mundo físico sin requerir un contacto previo. La integración de tecnologías como Firebase, Supabase y Nearby API ha resultado ser una elección acertada, ofreciendo tanto robustez como flexibilidad. Las pruebas realizadas, tanto técnicas como con usuarios reales, demuestran que la aplicación es estable, segura y fácil de usar.

Más allá de cumplir los objetivos iniciales, el desarrollo del proyecto ha permitido también adquirir experiencia práctica en gestión de errores, compatibilidad entre dispositivos, uso eficiente de APIs modernas y diseño de interfaces móviles. Estos resultados consolidan el proyecto como una solución completa, con posibilidad real de ser publicada, mantenida y mejorada en el futuro.

6. Conclusiones

Al finalizar el desarrollo de este proyecto, podemos afirmar que se han cumplido los objetivos planteados inicialmente, tanto a nivel general como en los específicos establecidos al comienzo de la memoria. El resultado final es una aplicación funcional, coherente y técnicamente sólida, que responde a una necesidad real y que ofrece una propuesta novedosa en el ámbito de las aplicaciones sociales basadas en proximidad física.

6.1. Cumplimiento de los objetivos

- **Objetivo general:** Se ha logrado desarrollar una aplicación móvil que permite a los usuarios conectar con personas cercanas sin necesidad de haber tenido contacto previo, promoviendo encuentros espontáneos pero seguros. El sistema protege la privacidad, da control al usuario y presenta una alternativa moderna y funcional para conocer nuevas personas.

- **Objetivos específicos:**
    - Interfaz visual e intuitiva: Aunque en el anteproyecto se planteó la implementación directa de interfaces diseñadas en Figma, durante el desarrollo se decidió rediseñar directamente en Jetpack Compose para garantizar la compatibilidad y rendimiento. El resultado ha sido una interfaz clara, atractiva y funcional.
    - Detección de usuarios cercanos: Utilizando la API Nearby de Google, se consiguió una detección eficaz de otros usuarios a distancias reales, con actualizaciones dinámicas en tiempo real.
    - Perfiles personalizables: Se implementó la posibilidad de editar imágenes, redes sociales y datos personales. La información se gestiona de forma segura con Firebase y Supabase.
    - Sistema de autenticación robusto: Con Firebase Authentication, se logró un acceso controlado, validaciones estrictas y persistencia de sesión.
    - Sistema de amistad y mensajería: Los usuarios pueden enviarse solicitudes, formar amistades y mantener conversaciones privadas, con gestión automática de relaciones y eliminación de datos cuando se cancela una conexión.

6.2. Evolución del proyecto y adaptación del anteproyecto

Es importante destacar que a lo largo del desarrollo del proyecto se han introducido múltiples modificaciones respecto al anteproyecto inicial. Estas decisiones no fueron improvisadas, sino fruto de un enfoque iterativo basado en pruebas constantes, retroalimentación real de usuarios y mejoras de rendimiento.

Entre los cambios más significativos se encuentran:

- El rediseño de interfaces inicialmente previstas en Figma, que fueron adaptadas directamente en Jetpack Compose tras detectar problemas de compatibilidad.

- La reorganización de la arquitectura de la app para mejorar la modularidad del código y la reutilización de componentes.

- La mejora progresiva del sistema de gestión de permisos, detección de dispositivos y validación de formularios, que inicialmente eran más simples.

- La inclusión de medidas de seguridad adicionales, como control de unicidad en usuarios y restricciones en Firebase para evitar accesos no autorizados.

Estas modificaciones han contribuido significativamente a que el proyecto haya evolucionado hacia una solución más robusta, optimizada y alineada con las buenas prácticas del desarrollo profesional de aplicaciones móviles.

6.3. Valor técnico y aprendizaje

El desarrollo de esta aplicación ha requerido un trabajo multidisciplinar que ha permitido aplicar y reforzar conocimientos clave del ciclo formativo. Se ha trabajado con tecnologías punteras como:

- Firebase (Authentication, Firestore, Realtime Database)

- Supabase para gestión de archivos

- Nearby API para conexión entre dispositivos

- Jetpack Compose para interfaces modernas

- GitHub para control de versiones

Todo ello ha exigido una profunda comprensión de Android, trabajo con APIs, validación de datos, gestión de sesiones, notificaciones y pruebas en distintos entornos. Además, el trabajo en equipo, la organización de tareas y la toma de decisiones técnicas han sido clave para el éxito del proyecto.

6.4. Limitaciones y posibles mejoras

Aunque el resultado ha sido muy positivo, también se han detectado puntos que podrían mejorarse en futuras versiones:

- Optimización de la detección en espacios con muchas interferencias.

- Inclusión de un sistema de reportes y moderación de usuarios.

- Implementación de perfiles verificados o control de contenido inapropiado.

- Mejora del rendimiento en dispositivos de gama baja.

- Versión multilingüe para ampliar el público potencial.

6.5. Conclusión final

Este proyecto ha sido una experiencia completa y enriquecedora tanto en lo personal como en lo técnico. No solo hemos cumplido los objetivos planteados, sino que también hemos aprendido a adaptar el desarrollo a los cambios necesarios, a tomar decisiones fundamentadas y a enfrentarnos a problemas reales con soluciones efectivas.

En resumen, este trabajo representa no solo un proyecto académico, sino una solución real, con potencial de crecimiento y utilidad, que demuestra la capacidad de aplicar en la práctica todo lo aprendido durante el ciclo formativo.